LECTURE 22

Logistic Regression I

Moving from regression to classification.

Goals for this Lecture

Moving away from linear regression – it's time for a new type of model

- Introduce a new task: classification
- Deriving a new model to handle this task

Agenda

- Regression vs. Classification
- The Logistic Regression Model
- Cross-Entropy Loss

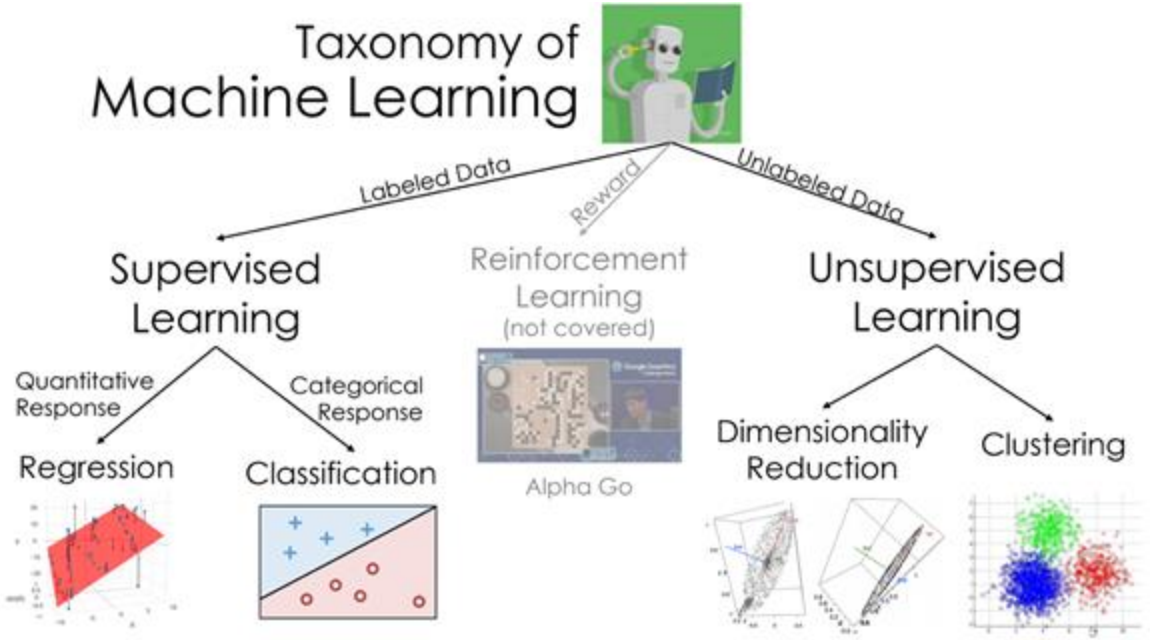
Regression vs Classification

- **Regression vs. Classification**
- The Logistic Regression Model
- Cross-Entropy Loss

Beyond Regression

Up until this point, we have been working exclusively with linear regression.

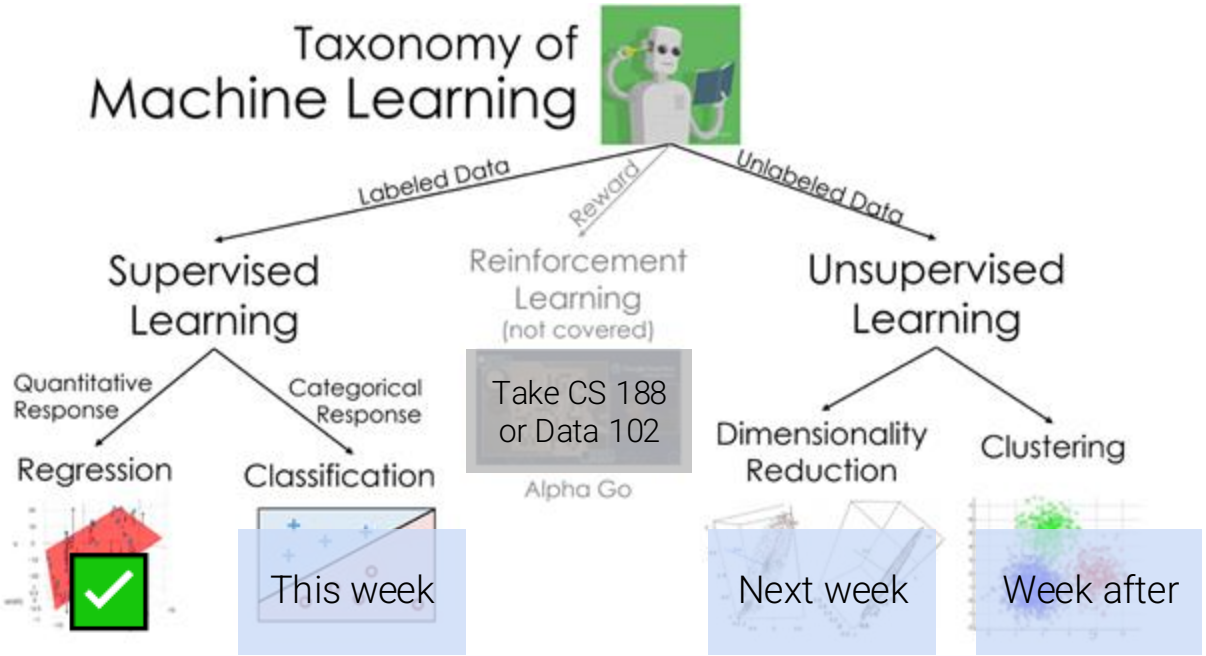
In the machine learning “landscape,” there are many other types of models.



Beyond Regression

Up until this point, we have been working exclusively with linear regression.

In the machine learning "landscape," there are many other types of models.

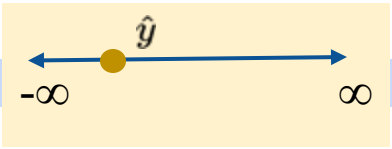


So Far: Regression

In regression, we use unbounded numeric features to predict an *unbounded numeric output*.

GAME_ID	TEAM_NAME	MATCHUP	REB	FTM	TOV	GOAL_DIFF	WON
21700001	Boston Celtics	BOS @ CLE	46	19	12	-0.049	0
21700002	Golden State Warriors	GSW vs. HOU	41	19	17	0.053	0
21700003	Charlotte Hornets	CHA @ DET	47	23	17	-0.030	0
21700004	Indiana Pacers	IND vs. BKN	47	25	14	0.041	1
21700005	Orlando Magic	ORL vs. MIA	50	22	15	0.042	1

$x^\top \theta$



Input: numeric features

Model: linear combination

Output: numeric prediction

Examples:

- Predict goal difference from turnover %
- Predict tip from total bill
- Predict mpg from hp

Now: Classification

In **classification**, we use unbounded numeric features to predict a *categorical class*.

Examples:

- Predict *which team won* from turnover %
- Predict *day of week* from total bill
- Predict *model of car* from hp

GAME_ID	TEAM_NAME	MATCHUP	REB	FTM	TOV	GOAL_DIFF	WON
21700001	Boston Celtics	BOS @ CLE	46	19	12	-0.049	0
21700002	Golden State Warriors	GSW vs. HOU	41	19	17	0.053	0
21700003	Charlotte Hornets	CHA @ DET	47	23	17	-0.030	0
21700004	Indiana Pacers	IND vs. BKN	47	25	14	0.041	1
21700005	Orlando Magic	ORL vs. MIA	50	22	15	0.042	1

Input: numeric features

$$p = \sigma(x^\top \theta)$$

Model: linear combination transformed by non-linear **sigmoid**

Win?
If p > 0.5: predict a win
Other: predict a loss

Decision rule

Output: **class**

An aside: we will use logistic "regression" to perform a *classification* task. Here, "regression" refers to the type of model, not the task being performed.

Kinds of Classification

We are interested in predicting some **categorical variable**, or **response**, y.

Binary classification

- Two classes
- **Responses** y are either 0 or 1

win or lose

disease or no disease

spam or ham

Multiclass classification

- Many classes
- Examples: Image labeling (Pishi, Thor, Hera), next word in a sentence, etc.

Structured prediction tasks

- Multiple related classification predictions
- Examples: Translation, voice recognition, etc.

Our new goal: predict a **binary** output (y_hat = 0 or y_hat = 1) given inputted numeric features

The Modeling Process

	Regression ($y \in \mathbb{R}$)	**Classification** ($y \in \{0, 1\}$)
1. Choose a model	Linear Regression $\hat{y} = f_\theta(x) = x^T\theta$	??
2. Choose a loss function	Squared Loss or Absolute Loss	??
3. Fit the model	Regularization Sklearn/Gradient descent	Regularization Sklearn/Gradient descent
4. Evaluate model performance	R^2, Residuals, etc.	?? (next lecture)

The Logistic Regression Model

- Regression vs. Classification
- **The Logistic Regression Model**
- Cross-Entropy Loss

The `games` Dataset

New modeling task, new dataset.

The `games` dataset describes the win/loss results of basketball teams.

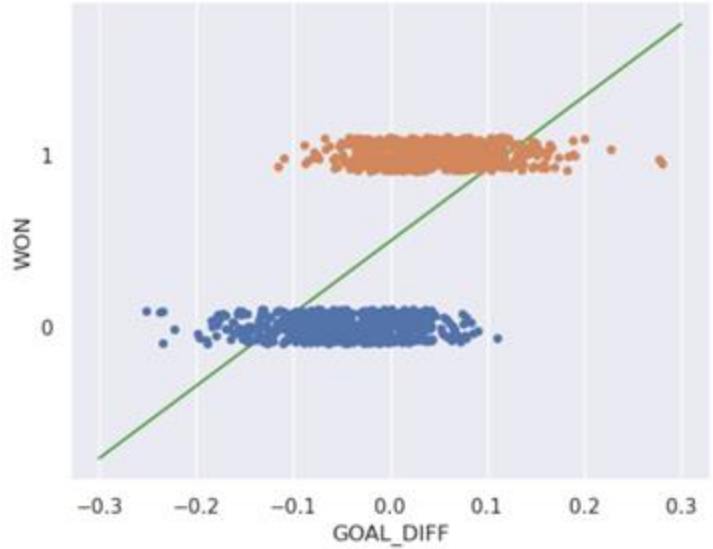
GAME_ID	TEAM_NAME	MATCHUP	WON	GOAL_DIFF
21700001	Boston Celtics	BOS @ CLE	0	-0.049
21700002	Golden State Warriors	GSW vs. HOU	0	0.053
21700003	Charlotte Hornets	CHA @ DET	0	-0.030
21700004	Indiana Pacers	IND vs. BKN	1	0.041
21700005	Orlando Magic	ORL vs. MIA	1	0.042

Difference in field goal success rate between teams

If a team won their game, we say they are in "Class 1"

Why Not Least Squares Linear Regression?

> I suppose it is tempting, if the only tool you have is a hammer, to treat everything as if it were a nail.
>
> – Abraham Maslow, *The Psychology of Science*



Problems:

- The output $\hat{y}$ can be outside the label range {0, 1}.
- Some outputs can't be interpreted: what does a class of "-2.3" mean?

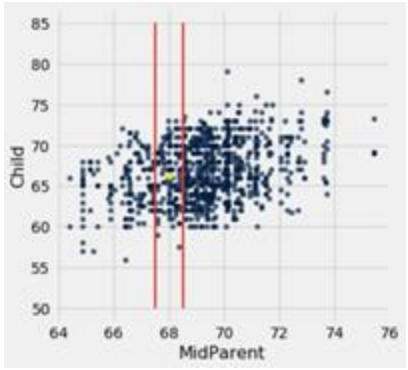
Demo

Back to Basics: the Graph of Averages

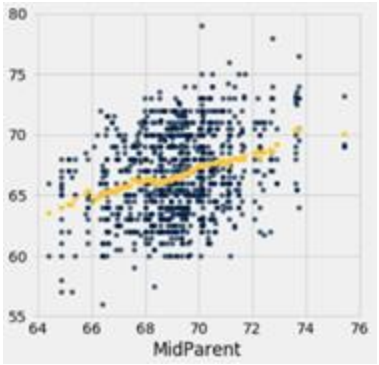
Clearly, least squares regression won't work here. We need to start fresh.

In Data 8 book, you built up to the idea of linear regression by considering the **graph of averages**.

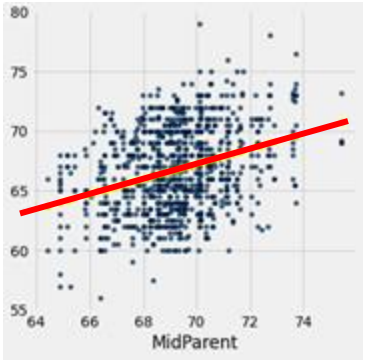
> For an input x, compute the **average value of y for all nearby x**, and predict that.
>
> [Data 8 textbook]



Bucket x-axis into bins



Average y vs. x bins is approximately linear



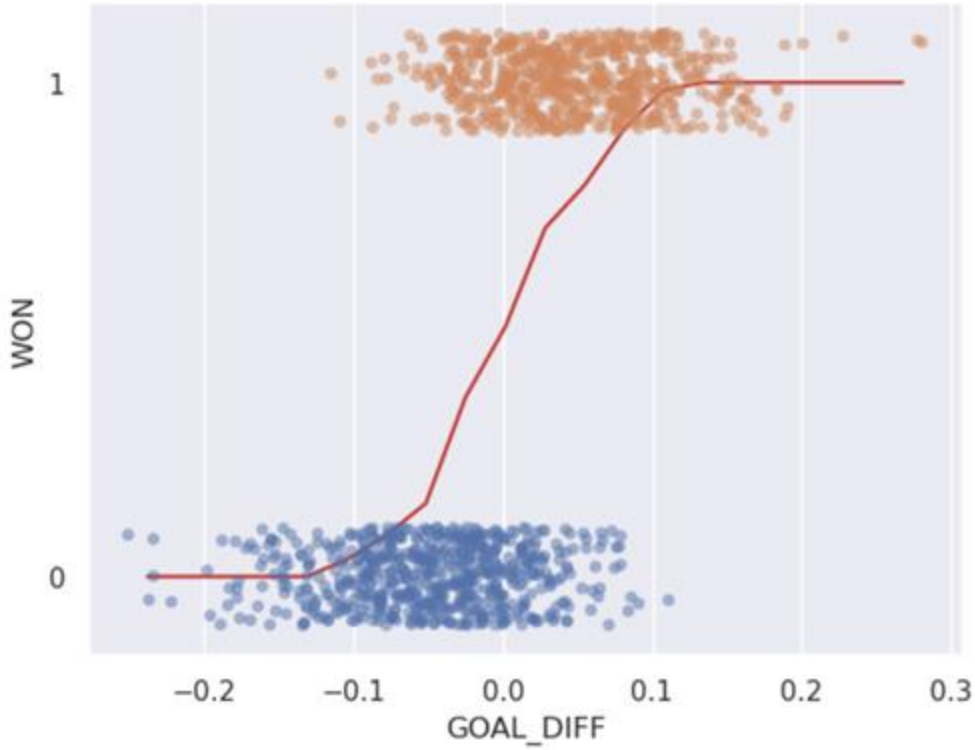
Parametric linear model

$$\hat{y} = x^T\theta$$

Graph of Averages for Our Classification Task

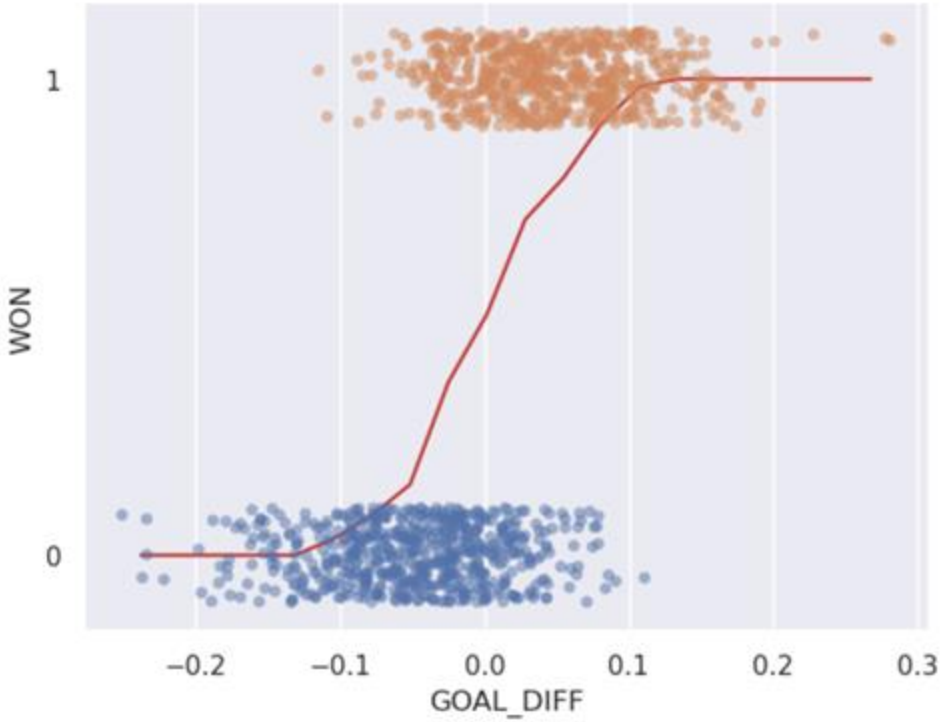
> For an input x, compute the **average value of y for all nearby x**, and predict that.

[textbook]



Demo

Graph of Averages for Our Classification Task



Looking a lot better!

Some observations:

- All predictions are between 0 and 1.
- Our predictions are **non-linear.**
  - Specifically, we see an “S” shape.

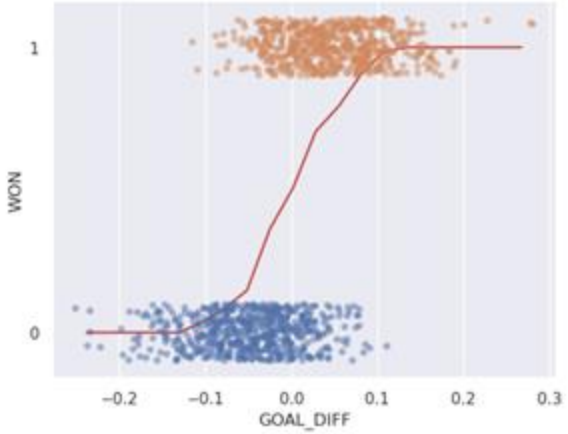
    This will be important soon.

Graph of Averages for Our Classification Task

Thinking more deeply about what we've just done.

To compute the average of a bin, we:

- Counted the number of wins in the bin.
- Divided by the total number of datapoints in the bin.



$$\frac{1(\#\, Y{=}1 \text{ in bin}) + 0(\#\, Y{=}0 \text{ in bin})}{\#\text{ datapoints in bin}} = \frac{\#\, Y{=}1 \text{ in bin}}{\#\text{ datapoints in bin}} = P(Y = 1 \,|\, \text{bin})$$

This is the probability that Y is 1 in that bin!

Predicting Probabilities

Our curve is modeling the *probability* that Y = 1 for a particular value of x.

$$p = P(Y = 1 \mid x)$$

- This matches our observation that all predictions are between 0 and 1 – the predictions are representing probabilities.

New modeling goal: model the *probability* of a datapoint belonging to Class 1.

Handling the Non-Linear Output

New modeling goal: model the *probability* of a datapoint belonging to Class 1.

We still seem to have a problem: our probability curve is non-linear, but we only know linear modeling techniques.

Good news: we've seen this before! To capture non-linear relationships, we:

1. Applied transformations to linearize the relationship.
2. Fit a linear model to the transformed variables.
3. Transformed the variables back to find their underlying relationship.

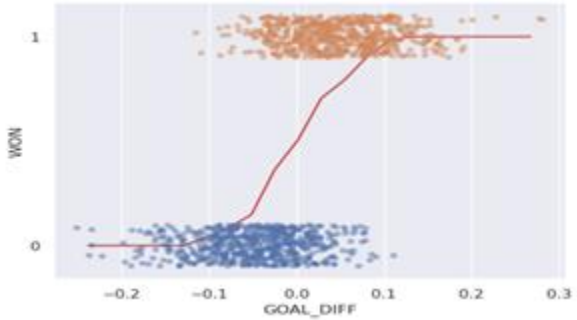
We'll use the exact same process here.

Step 1: Linearize the Relationship

Our S-shaped probability curve doesn't match any relationship we've seen before. The bulge diagram won't help here.

To understand what transformation to apply, think about our eventual goal: assign each datapoint to its *most likely* class.

How do we decide which class is more likely? One way: check which class has the higher predicted probability.

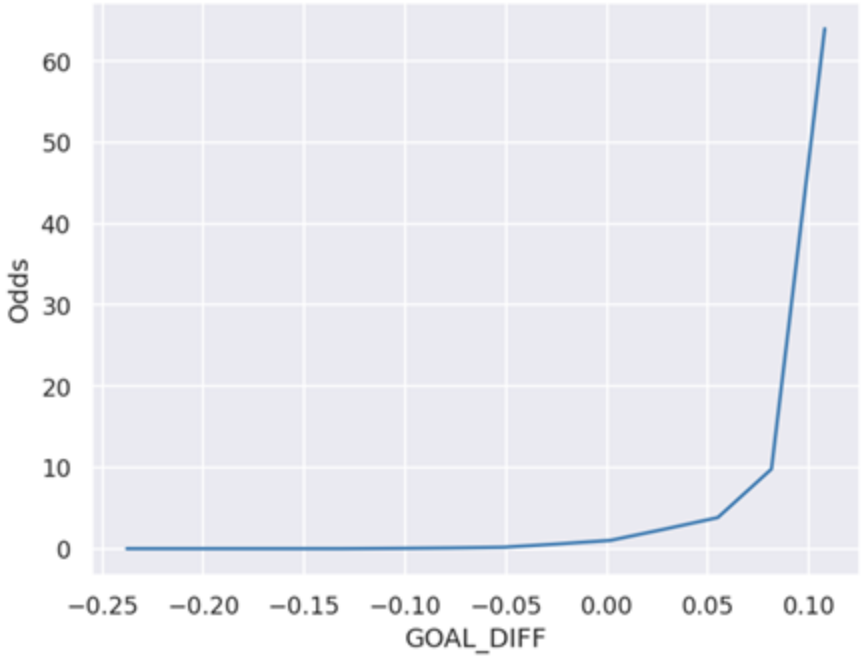


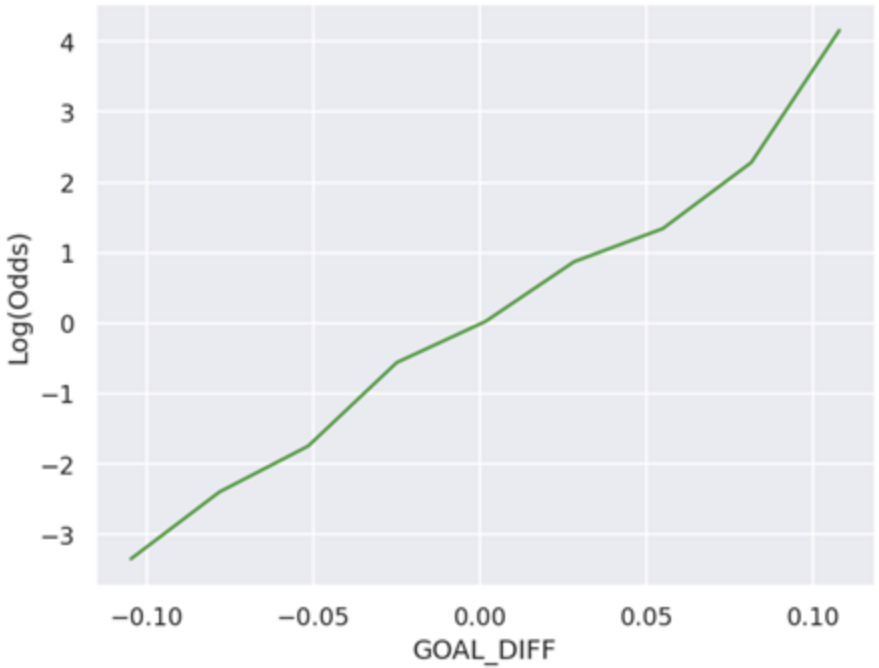
"Odds" is defined as the ratio of the probability of Y being Class 1 to the probability of Y being Class 0

$$\text{odds} = \frac{P(Y=1\,|\,x)}{P(Y=0\,|\,x)} = \frac{p}{1-p}$$

Step 1: Linearize the Relationship

The odds curve looks roughly exponential. To linearize it, we'll take the logarithm*.

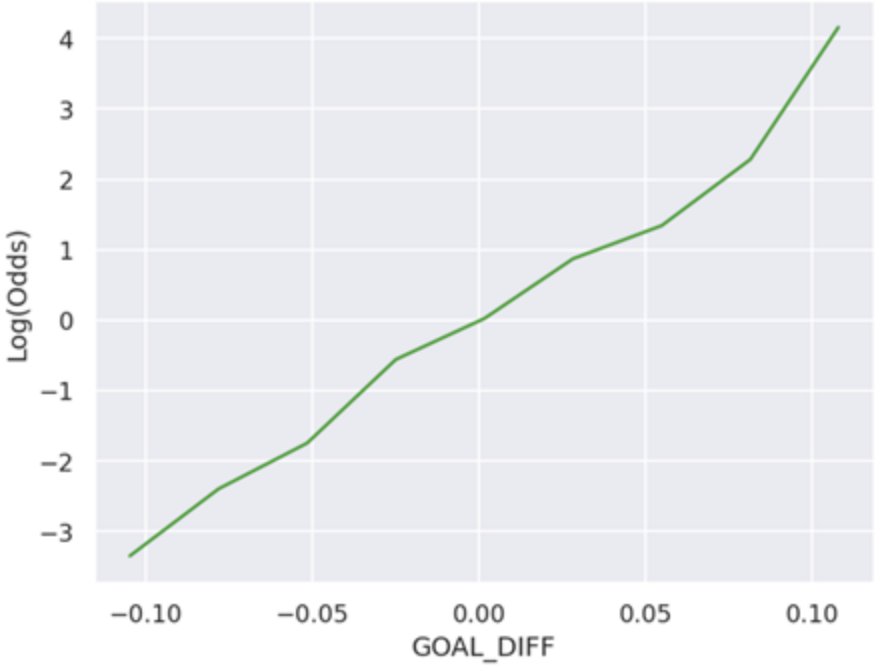




*In Data 100: assume that "log" means base e natural log unless told otherwise.

Step 2: Find a Linear Combination of the Features

We now have a linear relationship between our transformed variables. We can represent this relationship as a linear combination of the features.



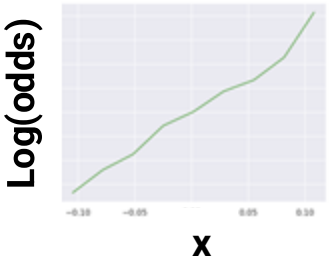
Our linear combination

$$z = x^\top \theta = \theta_0 + \theta_1 x_1 + \ldots + \theta_p x_p$$

Remember that our goal is to predict the *probability* of Class 1. This linear combination isn't our final prediction! We use z instead of y_hat to remind ourselves.

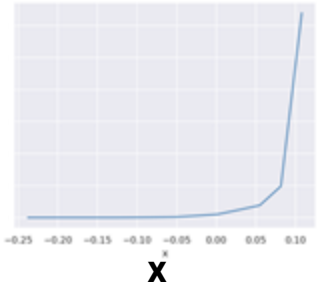
$$z = \log(\text{odds}) = \log\left(\frac{p}{1-p}\right)$$

Step 3: Transform Back to Original Variables

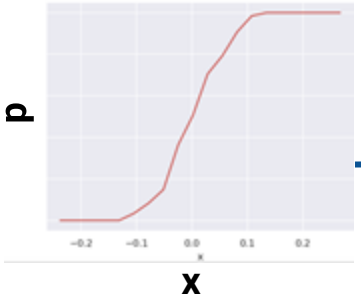


Solve for p:

$$\log\left(\frac{p}{1-p}\right) = x^T\theta$$



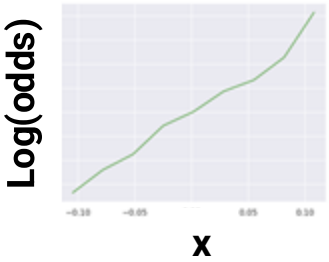
$$\frac{p}{1-p} = e^{x^T\theta}$$



$$p = \frac{1}{1+e^{-x^T\theta}}$$

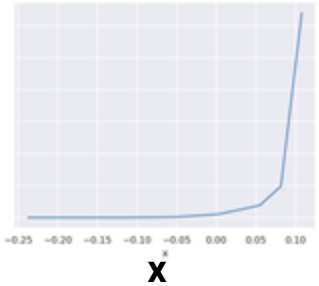
This is called the **logistic function**, $\sigma(\ )$.

Step 3: Transform Back to Original Variables



Solve for p:

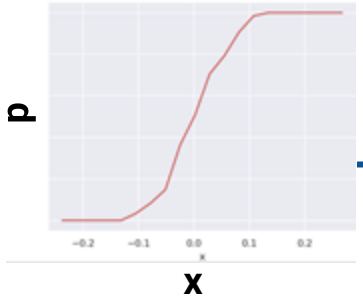
$$\log\left(\frac{p}{1-p}\right) = x^T\theta$$



$$\frac{p}{1-p} = e^{x^T\theta}$$

$$p = e^{x^T\theta} - pe^{x^T\theta}$$

$$p = \frac{e^{x^T\theta}}{1+e^{x^T\theta}}$$

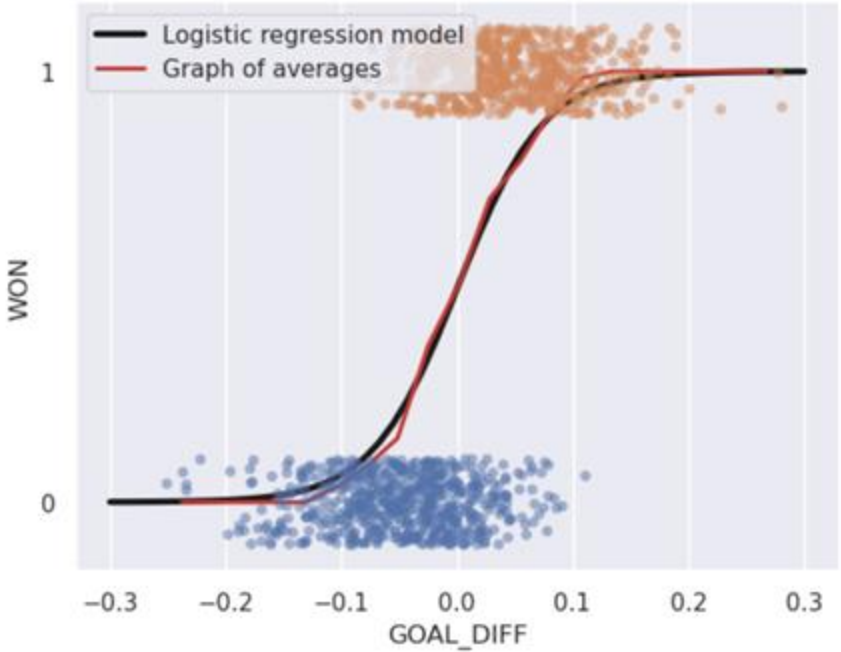


$$p = \frac{1}{1+e^{-x^T\theta}}$$

This is called the **logistic function**, $\sigma(\ )$.

Arriving at the Logistic Regression Model

We have just derived the **logistic regression model** for the probability of a datapoint belonging to Class 1.



$$P(Y=1\,|\,x) = \frac{1}{1+e^{-x^\top\theta}}$$

$$= \frac{1}{1+e^{-(\theta_0+\theta_1 x_1+\ldots+\theta_p x_p)}}$$

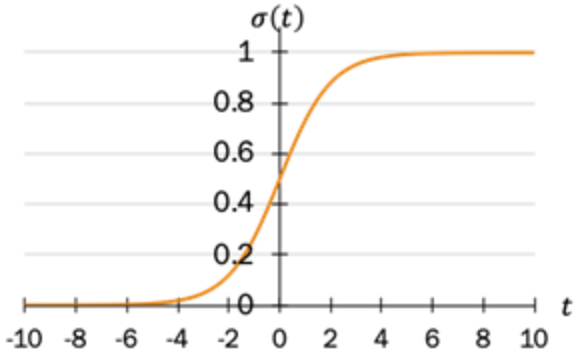
To predict a probability:

- Compute a linear combination of the features, $x^\top\theta$
- Apply the **sigmoid function** $\sigma(x^\top\theta)$

The Sigmoid Function

The S-shaped curve we derived is formally known as the **sigmoid function**.

$$\sigma(t) = \frac{1}{1 + e^{-t}}$$



Reflection/ Symmetry $1 - \sigma(t) = \frac{e^{-t}}{1 + e^{-t}} = \sigma(-t)$

Inverse $t = \sigma^{-1}(p) = \log\left(\frac{p}{1-p}\right)$

Domain $-\infty < t < \infty$

Range $0 < \sigma(t) < 1$

Derivative

$$\frac{d}{dt}\sigma(t) = \sigma(t)(1 - \sigma(t)) = \sigma(t)\sigma(-t)$$

The Sigmoid Converts Numerical Features to Probabilities

GAME_ID	TEAM_NAME	MATCHUP	REB	FTM	TOV	GOAL_DIFF	WON
21700001	Boston Celtics	BOS @ CLE	46	19	12	-0.049	0
21700002	Golden State Warriors	GSW vs. HOU	41	19	17	0.053	0
21700003	Charlotte Hornets	CHA @ DET	47	23	17	-0.030	0
21700004	Indiana Pacers	IND vs. BKN	47	25	14	0.041	1
21700005	Orlando Magic	ORL vs. MIA	50	22	15	0.042	1

Input: numeric features

$$p = \sigma(x^\top \theta)$$

Model: linear combination transformed by **activation function**

Next lecture:

Win?
If p > 0.5: predict a win
Other: predict a loss

Decision rule

Output: **class**

In logistic regression, the sigmoid transforms a linear combination of numerical features into a **probability.**

$$p = \sigma(x^\top \theta)$$

Formalizing the Logistic Regression Model

Our main takeaways of this section:

- Fit the "S" curve as best as possible.
- The curve models probability: P(Y = 1 | x).
- Assume log-odds is a linear combination of x and θ.

Putting it all together:

$$\hat{P}_\theta(Y = 1|x) = \frac{1}{1 + e^{-x^T\theta}}$$

Estimated probability that given the features x, the response is 1

Logistic function $\sigma(\ )$ at the value $x^T\theta$

The logistic regression model is most commonly written as follows:

$$\hat{P}_\theta(Y = 1|x) = \sigma(x^T\theta)$$

Looks like linear regression. Now wrapped with $\boldsymbol{\sigma}(\ )$!

Example Calculation

Suppose I want to predict the probability that a team wins a game, given **GOAL_DIFF** (first feature) and **number of free throws** (second feature).

I fit a logistic regression model (with no intercept) using my training data, and estimate the optimal parameters:

$$\hat{\theta}^T = [0.1 \quad -0.5]$$

Now, you want to predict the probability that a new team wins their game.

$$x^T = [15 \quad 1]$$

slido

Suppose x^T contains the GOAL_DIFF and number of free throws for a new team. What is the predicted probability that this new team will win?

Example Calculation

Suppose I want to predict the probability that a team wins a game, given **GOAL_DIFF** (first feature) and **number of free throws** (second feature).

I fit a logistic regression model (with no intercept) using my training data, and estimate the optimal parameters:

$$\hat{\theta}^T = [0.1 \quad -0.5]$$

Now, you want to predict the probability that a new team wins their game.

$$x^T = [15 \quad 1]$$

$$\begin{aligned}\hat{P}_{\hat{\theta}}(Y = 1|x) &= \sigma(x^T\hat{\theta}) \\ &= \sigma(0.1 \cdot 15 + (-0.5) \cdot 1) \\ &= \sigma(1) \\ &= \frac{1}{1 + e^{-1}} \\ &\approx 0.7311\end{aligned}$$

Because the response is more likely to be 1 than 0, a reasonable prediction is

$$\hat{y} = 1$$

(more on this next lecture)

Properties of the Logistic Model

Consider a logistic regression model with one feature and an intercept term (Desmos):

$$p = P(Y = 1 \mid x) = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x)}}$$

Properties:

- θ_0 controls the position of the curve along the horizontal axis.
- The magnitude of θ_1 controls the "steepness" of the sigmoid.
- The sign of θ_1 controls the orientation of the curve.

Interlude

Classification is hard.

Cross-Entropy Loss

- Regression vs. Classification
- The Logistic Regression Model
- **Cross-Entropy Loss**

The Modeling Process

	Regression ($y \in \mathbb{R}$)	Classification ($y \in \{0, 1\}$)
1. Choose a model	Linear Regression $\hat{y} = f_\theta(x) = x^T\theta$	Logistic Regression $\hat{P}_\theta(Y = 1\|x) = \sigma(x^T\theta)$
2. Choose a loss function	Squared Loss or Absolute Loss	??
3. Fit the model	Regularization Sklearn/Gradient descent	Regularization Sklearn/Gradient descent
4. Evaluate model performance	R^2, Residuals, etc.	?? (next time)

The Modeling Process

	Regression ($y \in \mathbb{R}$)	Classification ($y \in \{0, 1\}$)
1. Choose a model ✓	Linear Regression $\hat{y} = f_\theta(x) = x^T\theta$	Logistic Regression $\hat{P}_\theta(Y = 1\|x) = \sigma(x^T\theta)$
2. Choose a loss function	Squared Loss or Absolute Loss	Can squared loss still work?
3. Fit the model	Regularization Sklearn/Gradient descent	Regularization Sklearn/Gradient descent
4. Evaluate model performance	R^2, Residuals, etc.	?? (next time)

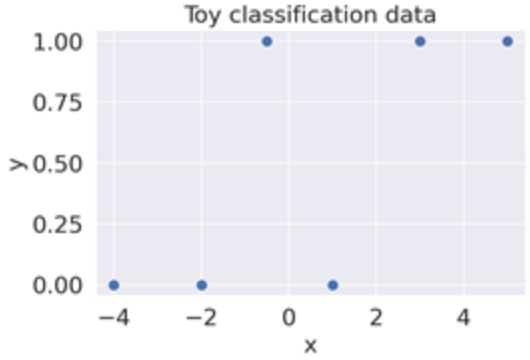
Toy Dataset: L2 Loss

Logistic Regression model:

$$\hat{P}_\theta(Y = 1|x) = \sigma(x^T\theta)$$

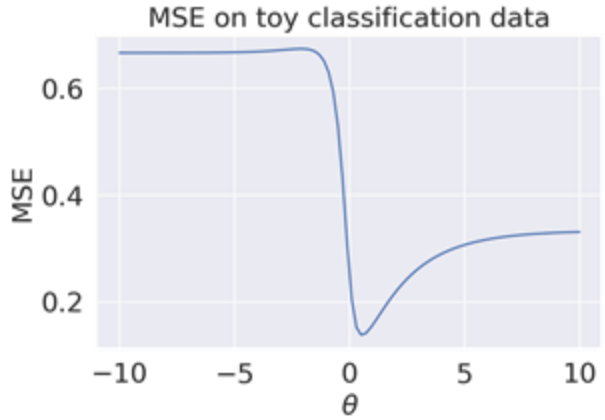
Assume no intercept.
So x, θ both scalars.

	x	y
0	-4.0	0
1	-2.0	0
2	-0.5	1
3	1.0	0
4	3.0	1
5	5.0	1



Mean Squared Error:

$$R(\theta) = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \sigma(x_i^T\theta)\right)^2$$



The MSE loss surface for logistic regression has many issues!

Demo

slido

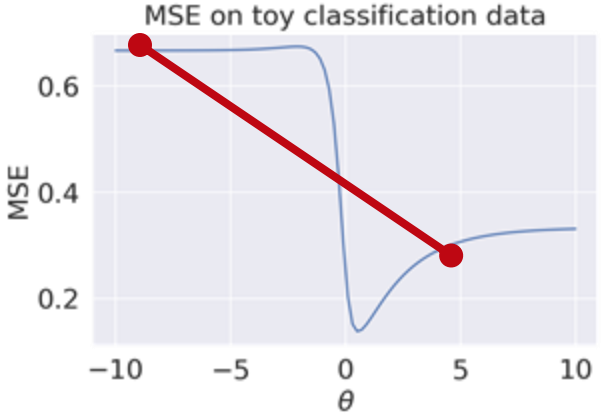
What problems might arise from using MSE loss with logistic regression?

Pitfalls of Squared Loss

$$R(\theta) = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \sigma(x_i^T\theta)\right)^2$$

1. Non-convex. Gets stuck in local minima.

Secant line crosses function, so $R''(\theta)$ is not greater than 0 for all θ.



Demo

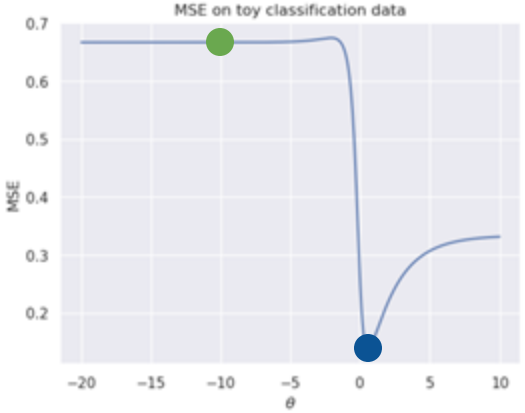
Pitfalls of Squared Loss

$$R(\theta) = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \sigma(x_i^T\theta)\right)^2$$

1. Non-convex. Gets stuck in local minima.

Secant line crosses function, so $R''(\theta)$ is not greater than 0 for all θ.

Gradient Descent: Different initial guesses will yield different optimal estimates.




```
from scipy.optimize import minimize

minimize(mse_loss_toy_nobias, x0 = 0)["x"][0]
```

```
0.5446601825581691
```

```
minimize(mse_loss_toy_nobias, x0 = -5)["x"][0]
```

```
-10.343653061026611
```

Demo

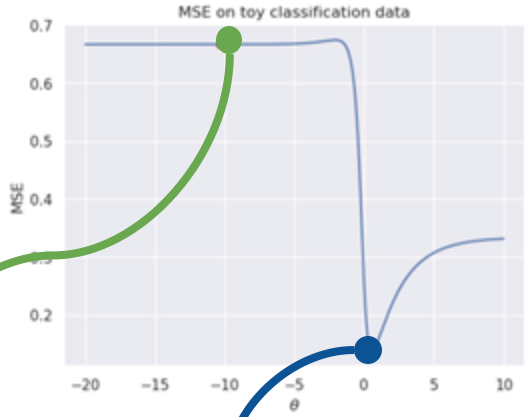
Pitfalls of Squared Loss

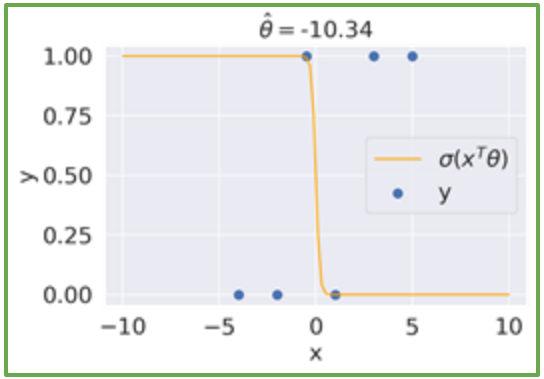
$$R(\theta) = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \sigma(x_i^T\theta)\right)^2$$

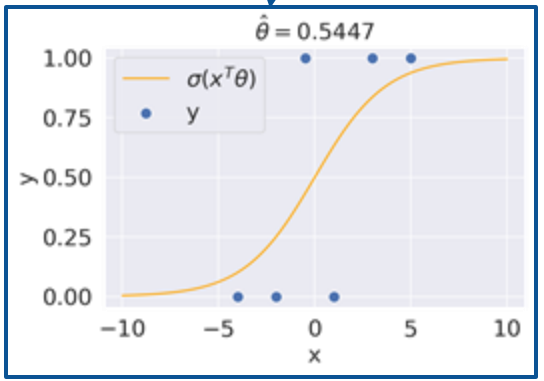
1. Non-convex. Gets stuck in local minima.

Secant line crosses function, so $R''(\theta)$ is not greater than 0 for all θ.

Gradient Descent: Different initial guesses will yield different optimal estimates.







Demo

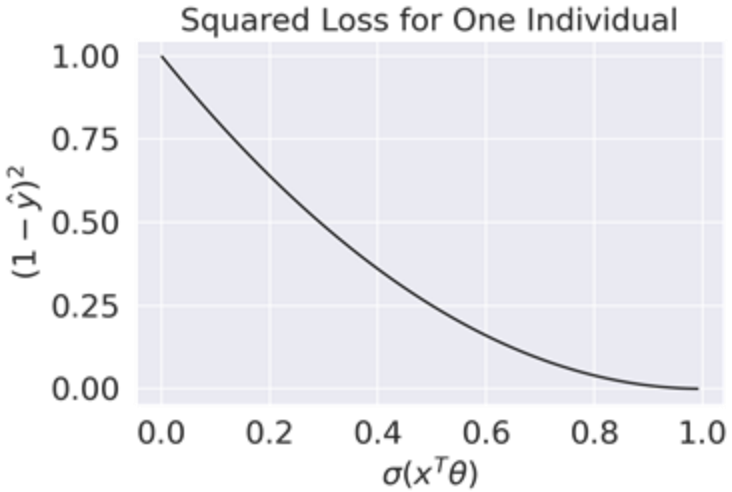
Pitfalls of Squared Loss

$$R(\theta) = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \sigma(x_i^T\theta)\right)^2$$

1. **Non-convex**. Gets stuck in local minima.

2. **Bounded**. Not a good measure of model error.

- We'd like loss functions to penalize "off" predictions.
- MSE never gets very large, because both response and predicted probability are bounded by 1.



If true y=1 but predicted probability = 0:

$$(y-p)^2 = (1-0)^2 = 1$$

Demo

Choosing a Different Loss Function

	Regression ($y \in \mathbb{R}$)	Classification ($y \in \{0, 1\}$)
1. Choose a model ✓	Linear Regression $\hat{y} = f_\theta(x) = x^T\theta$	Logistic Regression $\hat{P}_\theta(Y = 1\|x) = \sigma(x^T\theta)$
2. Choose a loss function	Squared Loss or Absolute Loss	**Cross-Entropy Loss**
3. Fit the model	Regularization Sklearn/Gradient descent	Regularization Sklearn/Gradient descent
4. Evaluate model performance	R^2, Residuals, etc.	?? (next time)

Loss in Classification

Let y be a binary label {0, 1}, and p be the model's predicted probability of the label being 1.

In a classification task, how do we want our loss function to behave?

- When the true y is 1, we should incur *low* loss when the model predicts large p.
- When the true y is 0, we should incur *high* loss when the model predicts large p.

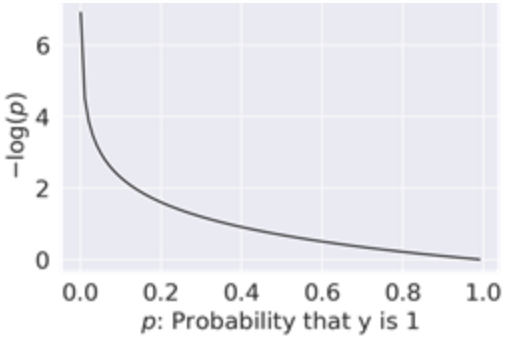
In other words, the behavior we need from our loss function depends on the value of the true class, y.

Cross-Entropy Loss

Let y be a binary label {0, 1}, and p be the probability of the label being 1.

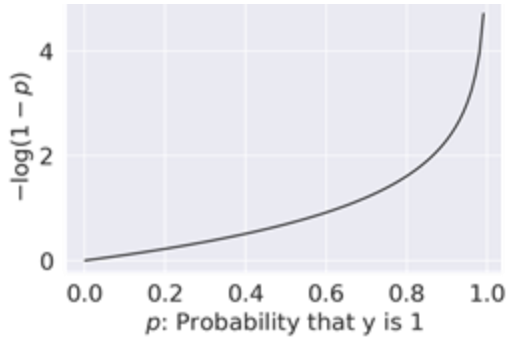
The **cross-entropy loss** is defined as:

$$\text{CE loss} = \begin{cases} -\log(p), & \text{if } y = 1 \\ -\log(1-p), & \text{if } y = 0 \end{cases}$$



For y = 1,

- p → 0: ∞ loss
- p → 1: zero loss



For y = 0,

- p → 0: zero loss
- p → 1: ∞ loss

Cross-Entropy Loss: Two Loss Functions In One!

The piecewise loss function we introduced just then is difficult to optimize – we don't want to check "which" loss to use at each step of optimizing theta.

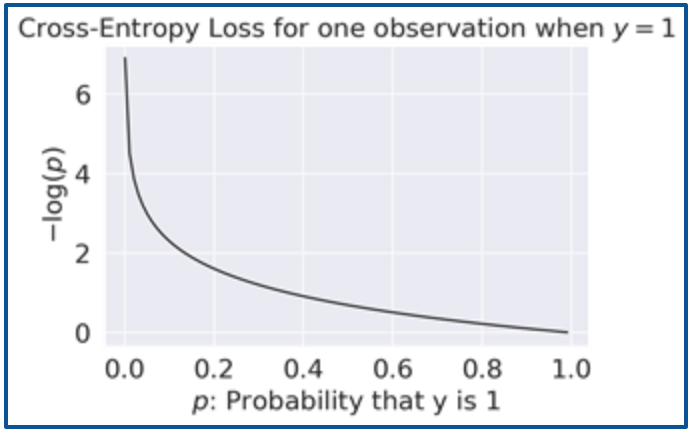
Cross-entropy loss can be equivalently expressed as:

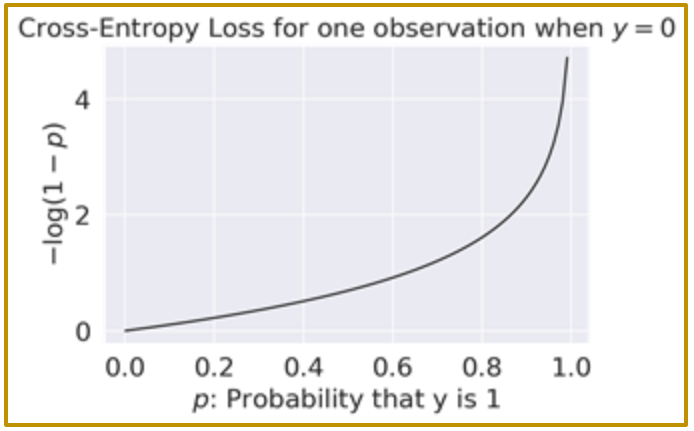
$$\text{CE loss} = \begin{cases} -\log(p), & \text{if } y = 1 \\ -\log(1-p), & \text{if } y = 0 \end{cases} \rightarrow -\left(y \log(p) + (1-y)\log(1-p)\right)$$

makes loss positive

for y = **1**, only this term stays

for y = **0**, only this term stays





Empirical Risk: Average Cross-Entropy Loss

For a single datapoint, the cross-entropy curve is convex. It has a global minimum.

$$-\left(y\log(p)+(1-y)\log(1-p)\right)$$

What about average cross-entropy loss, i.e., empirical risk?

For logistic regression, the empirical risk over a sample of size n is:

$$R(\theta) = -\frac{1}{n}\sum_{i=1}^{n}\left(y_i\log(p_i)+(1-y_i)\log(1-p_i)\right)$$

$$= -\frac{1}{n}\sum_{i=1}^{n}\left(y_i\log\left(\sigma\left(X_i^T\theta\right)\right)-(1-y_i)\log\left(1-\sigma\left(X_i^T\theta\right)\right)\right)$$

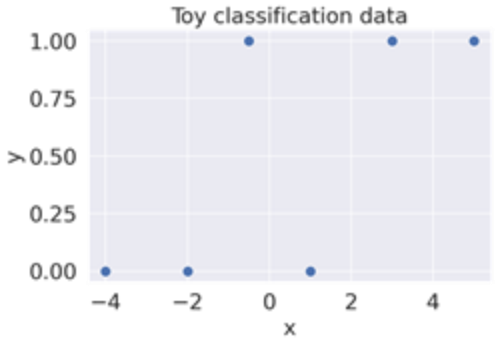
[Recall our model is $\hat{y}_i = p_i = \sigma\left(X_i^T\theta\right)$]

The optimization problem is therefore to find the estimate $\hat{\theta}$ that minimizes R(θ):

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \; -\frac{1}{n}\sum_{i=1}^{n}\left(y_i\log\left(\sigma\left(X_i^T\theta\right)\right)-(1-y_i)\log\left(1-\sigma\left(X_i^T\theta\right)\right)\right)$$

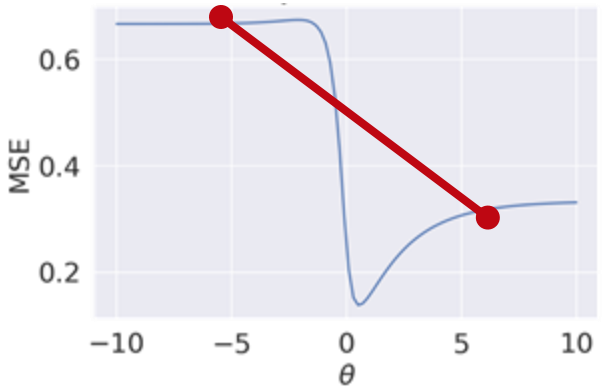
Convexity Proof By Picture

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \; -\frac{1}{n}\sum_{i=1}^{n}\left(y_i \log\left(\sigma\left(X_i^T\theta\right)\right) - (1-y_i)\log\left(1-\sigma\left(X_i^T\theta\right)\right)\right)$$



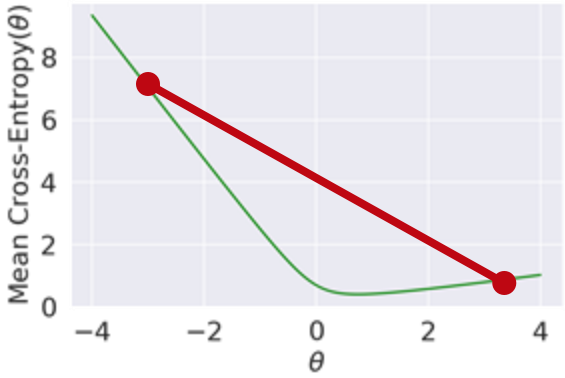
	x	y
0	-4.0	0
1	-2.0	0
2	-0.5	1
3	1.0	0
4	3.0	1
5	5.0	1

Squared Loss Surface



A straight line crosses the curve
Non-convex

Cross-Entropy Loss Surface



Convex!

LECTURE 22

Logistic Regression I

Bonus Material: Maximum Likelihood Estimation

It may have seemed like we just pulled cross-entropy loss out of thin air.

CE loss is justified by a probability analysis technique called maximum likelihood estimation. Read on if you would like to learn more.

Recorded walkthrough: link.

The No-Input Binary Classifier

Suppose you observed some outcomes of a coin (1 = Heads, 0 = Tails):

$$\{0, 0, 1, 1, 1, 1, 0, 0, 0, 0\}$$

Training data has only responses $\mathbb{Y}$ (no features $\mathbb{X}$)

For the next flip, do you predict heads or tails?

The No-Input Binary Classifier

Suppose you observed some outcomes of a coin (1 = Heads, 0 = Tails):

$$\{0, 0, 1, 1, 1, 1, 0, 0, 0, 0\}$$

Training data has only responses $\mathbb{Y}$ (no features $\mathbb{X}$)

For the next flip, do you predict heads or tails?

A reasonable model is to **assume all flips are IID** (i.e., same coin; same prob. of heads θ).

Parameter θ: Probability that flip == 1 (Heads)

Prediction: 1 or 0

1. Of the below, which is the best theta θ? Why?

A. 0.8 B. 0.5 C. 0.4
D. 0.2 E. Something else

2. For the next flip, would you predict 1 or 0?

The No-Input Binary Classifier

Suppose you observed some outcomes of a coin (1 = Heads, 0 = Tails):

$$\{0, 0, 1, 1, 1, 1, 0, 0, 0, 0\}$$

Training data has only responses $\mathbb{Y}$ (no features $\mathbb{X}$)

For the next flip, do you predict heads or tails?

A reasonable model is to **assume all flips are IID** (i.e., same coin; same prob. of heads θ).

Parameter θ: Probability that flip == 1 (Heads)

Prediction: 1 or 0

1. Of the below, which is the best theta θ? Why?

A. 0.8 **B.** 0.5 **C.** 0.4
D. 0.2 **E.** Something else

The No-Input Binary Classifier

Suppose you observed some outcomes of a coin (1 = Heads, 0 = Tails):

$$\{0, 0, 1, 1, 1, 1, 0, 0, 0, 0\}$$

Training data has only responses $\mathbb{Y}$ (no features $\mathbb{X}$)

For the next flip, do you predict heads or tails?

A reasonable model is to **assume all flips are IID** (i.e., same coin; same prob. of heads θ).

Parameter θ: Probability that flip == 1 (Heads)

Prediction: 1 or 0

1. Of the below, which is the best theta θ? Why?

A. 0.8 **B.** 0.5 **C. 0.4**

D. 0.2 **E.** Something else

0.4 is the most "intuitive" for two reasons:

1. Frequency of heads in our data.
2. Maximizes the **likelihood** of our data.

The No-Input Binary Classifier

Suppose you observed some outcomes of a coin (1 = Heads, 0 = Tails):

$$\{0, 0, 1, 1, 1, 1, 0, 0, 0, 0\}$$

Training data has only responses $\mathbb{Y}$ (no features $\mathbb{X}$)

For the next flip, do you predict heads or tails?

A reasonable model is to **assume all flips are IID** (i.e., same coin; same prob. of heads θ).

Parameter θ: Probability that flip == 1 (Heads)

Prediction: 1 or 0

1. Of the below, which is the best theta θ? Why?

A. 0.8 **B.** 0.5 **C.** **0.4**

D. 0.2 **E.** Something else

2. For the next flip, would you predict 1 or 0?

The most frequent outcome in the sample which is tails.

Likelihood of Data; Definition of Probability

A Bernoulli random variable Y with parameter p has distribution: $P(Y = y) = \begin{cases} p & \text{if } y = 1 \\ 1 - p & \text{if } y = 0 \end{cases}$

Given that all flips are IID from the same coin (probability of heads = p), the **likelihood** of our data is **proportional to** the probability of observing the datapoints.

Training data: [0, 0, 1, 1, 1, 1, 0, 0, 0, 0]

Data likelihood: $p^4(1-p)^6$.

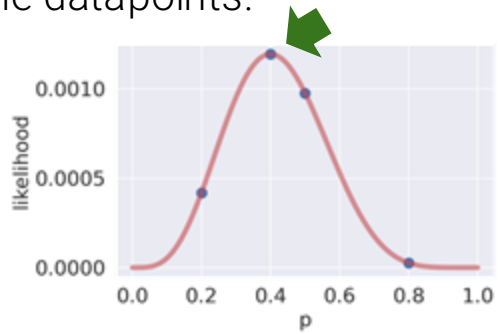
Likelihood of Data

A Bernoulli random variable Y with parameter p has distribution: $P(Y = y) = \begin{cases} p & \text{if } y = 1 \\ 1 - p & \text{if } y = 0 \end{cases}$

Given that all flips are IID from the same coin (probability of heads = p), the **likelihood** of our data is **proportional to** the probability of observing the datapoints.

Training data: [0, 0, 1, 1, 1, 1, 0, 0, 0, 0]

Data likelihood: $p^4(1-p)^6$.



An example of a bad estimate is parameter $p = 0.1$ since the likelihood of observing the training data is going to be :

$$(0.1)^4(0.9)^6 = 0.000053$$

An example of a bad estimate is parameter $p = 0.4$ since the likelihood of observing the training data is going to be :

$$(0.4)^4(0.6)^6 = 0.001194$$

Generalization of the Coin Demo

For training data: {0, 0, 1, 1, 1, 1, 0, 0, 0, 0}

0.4 is the most "intuitive" θ for two reasons:

1. Frequency of heads in our data
2. Maximizes the **likelihood** of our data:

$$\hat{\theta} = \underset{\theta}{\text{argmax}} \left(\theta^4 (1-\theta)^6\right)$$

Parameter θ:
Probability that
IID flip == 1 (Heads)

Prediction:
1 or 0

How can we generalize this notion of likelihood to **any** random binary sample?

$$\{y_1, y_2, \ldots, y_n\} \Rightarrow \hat{\theta} = \underset{\theta}{\text{argmax}} \left(???\right)$$

data (1's and 0's)

likelihood

A Compact Representation of the Bernoulli Probability Distribution

How can we generalize this notion of likelihood to **any** random binary sample?

$$\{y_1, y_2, \ldots, y_n\} \Rightarrow \hat{\theta} = \underset{\theta}{\text{argmax}}\,(???)$$

data (1's and 0's) likelihood

Let Y be Bernoulli(p). The probability distribution can be written compactly:

$$P(Y = y) = p^y (1-p)^{1-y}$$

For P(Y = **1**), only this term stays

For P(Y = **0**), only this term stays

(long, non-compact form):

$$P(Y = y) = \begin{cases} p & \text{if } y = 1 \\ 1-p & \text{if } y = 0 \end{cases}$$

Generalized Likelihood of Binary Data

How can we generalize this notion of likelihood to **any** random binary sample?

$$\{y_1, y_2, \ldots, y_n\} \Rightarrow \hat{\theta} = \underset{\theta}{\text{argmax}}\,(???)$$

data (1's and 0's) — likelihood

Let Y be Bernoulli(p). The probability distribution can be written compactly:

$$P(Y = y) = p^y(1-p)^{1-y}$$

For P(Y = **1**), only this term stays

For P(Y = **0**), only this term stays

If binary data are **IID with same** probability p, then the likelihood of the data is:

$$\prod_{i=1}^{n} p^{y_i}(1-p)^{(1-y_i)}$$

Ex: {0, 0, 1, 1, 1, 1, 0, 0, 0, 0} → $p^4(1-p)^6$

Generalized Likelihood of Binary Data

How can we generalize this notion of likelihood to any random binary sample?

$$\{y_1, y_2, \ldots, y_n\} \Rightarrow \hat{\theta} = \underset{\theta}{\operatorname{argmax}}\,(???)$$

data (1's and 0's) — likelihood

Let Y be Bernoulli(p). The probability distribution can be written compactly:

$$P(Y = y) = p^y (1-p)^{1-y}$$

For P(Y = **1**), only this term stays

For P(Y = **0**), only this term stays

If binary data are **IID with same** probability p, then the likelihood of the data is:

$$\prod_{i=1}^{n} p^{y_i} (1-p)^{(1-y_i)}$$

If data are independent with **different** probability p_i, then the likelihood of the data is:

(spoiler: for logistic regression, $p_i = \sigma(X_i^T \theta)$)

$$\prod_{i=1}^{n} p_i^{y_i} (1-p_i)^{(1-y_i)}$$

Maximum Likelihood Estimation (MLE)

Our **maximum likelihood estimation** problem:

- For i = 1, 2, …, n, let Y_i be independent Bernoulli(p_i). Observe data $\{y_1, y_2, \ldots, y_n\}$.
- We'd like to estimate $p_1, p_2, \ldots, p_n$.

Find $\hat{p}_1, \hat{p}_2, \ldots, \hat{p}_n$ that **maximize** $\prod_{i=1}^{n} p_i^{y_i}(1-p_i)^{(1-y_i)}$

Maximum Likelihood Estimation (MLE)

Our **maximum likelihood estimation** problem:

- For i = 1, 2, …, n, let Y_i be independent Bernoulli(p_i). Observe data $\{y_1, y_2, \ldots, y_n\}$.
- We'd like to estimate $p_1, p_2, \ldots, p_n$.

Find $\hat{p}_1, \hat{p}_2, \ldots, \hat{p}_n$ that **maximize** $\prod_{i=1}^{n} p_i^{y_i}(1-p_i)^{(1-y_i)}$

Equivalent, simplifying optimization problems (since we need to take the first derivative):

$$\underset{p_1, p_2, \ldots, p_n}{\text{maximize}} \quad \log\left(\prod_{i=1}^{n} p_i^{y_i}(1-p_i)^{(1-y_i)}\right)$$

(log is an increasing function. If a > b, then log(a) > log(b).)

$$= \sum_{i=1}^{n} \log\left(p_i^{y_i}(1-p_i)^{(1-y_i)}\right) = \sum_{i=1}^{n} \left(y_i \log(p_i) + (1-y_i)\log(1-p_i)\right)$$

Maximum Likelihood Estimation (MLE)

Our **maximum likelihood estimation** problem:

- For i = 1, 2, …, n, let Y_i be independent Bernoulli(p_i). Observe data $\{y_1, y_2, \ldots, y_n\}$.
- We'd like to estimate $p_1, p_2, \ldots, p_n$.

Find $\hat{p}_1, \hat{p}_2, \ldots, \hat{p}_n$ that **maximize** $\prod_{i=1}^{n} p_i^{y_i}(1-p_i)^{(1-y_i)}$

Equivalent, simplifying optimization problems (since we need to take the first derivative):

maximize $p_1, p_2, \ldots, p_n$ $\log\left(\prod_{i=1}^{n} p_i^{y_i}(1-p_i)^{(1-y_i)}\right)$ (log is an increasing function. If a > b, then log(a) > log(b).)

$$= \sum_{i=1}^{n} \log\left(p_i^{y_i}(1-p_i)^{(1-y_i)}\right) = \sum_{i=1}^{n} \left(y_i \log(p_i) + (1-y_i)\log(1-p_i)\right)$$

minimize $p_1, p_2, \ldots, p_n$ $-\frac{1}{n}\sum_{i=1}^{n} \left(y_i \log(p_i) + (1-y_i)\log(1-p_i)\right)$

Maximizing Likelihood == Minimizing Average Cross-Entropy

maximize $_{p_1, p_2, \ldots, p_n}$ $\prod_{i=1}^{n} p_i^{y_i}(1-p_i)^{(1-y_i)}$

Log is increasing; max/min properties

minimize $_{p_1, p_2, \ldots, p_n}$ $-\frac{1}{n}\sum_{i=1}^{n}\left(y_i \log(p_i) + (1-y_i)\log(1-p_i)\right)$

For logistic regression, let $p_i = \sigma(X_i^T\theta)$

minimize $_{\theta}$ $\underbrace{-\frac{1}{n}\sum_{i=1}^{n}\left(y_i \log(\sigma(X_i^T\theta)) + (1-y_i)\log(1-\sigma(X_i^T\theta))\right)}$

Cross-Entropy Loss!!

Minimizing cross-entropy loss is equivalent to **maximizing the likelihood of the training data**.

- We are choosing the model parameters that are "most likely", given this data.

Assumption: all data drawn **independently** from the same logistic regression model with parameter θ

- It turns out that many of the model + loss combinations we've seen can be motivated using MLE (OLS, Ridge Regression, etc.)
- You will study MLE further in probability and ML classes. But now you know it exists.

We Did it!

	Regression ($y \in \mathbb{R}$)	Classification ($y \in \{0, 1\}$)
1. Choose a model ✓	Linear Regression $\hat{y} = f_\theta(x) = x^T\theta$	Logistic Regression $\hat{P}_\theta(Y = 1\|x) = \sigma(x^T\theta)$
2. Choose a loss function ✓	Squared Loss or Absolute Loss	Average Cross-Entropy Loss $-\frac{1}{n}\sum_{i=1}^{n}\left(y_i \log(\sigma(X_i^T\theta) + (1 - y_i)\log(1 - \sigma(X_i^T\theta))\right)$
3. Fit the model	Regularization Sklearn/Gradient descent	Regularization Sklearn/Gradient descent
4. Evaluate model performance	R^2, Residuals, etc.	?? (next time)